FineRR-ZNS: Enabling Fine-Granularity Read Refreshing for ZNS SSDs

Jun Li[1], Zhibing Sha[2], Fan Yang[2], Xiaofei Xu[3], Xiaobai Chen[1], Jieming Yin[1*], Jianwei Liao[2*]
[1]School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China
[2]College of Computer and Information Science, Southwest University, Chongqing 400715, China
[3]School of Computing Technologies, RMIT University, Melbourne 3001, Australia
Emails: {junli, chenxb86, jieming.yin}@njupt.edu.cn, {zhibing.sha, fanyang0329, liaotoad}@gmail.com, xiaofei.xu@ieee.org

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132791

***Abstract*—Zoned namespace (ZNS) SSDs are emerging storage devices offering low cost, high performance, and software definability. By adopting host-managed zone-based sequential programming, ZNS SSDs effectively eliminate the space overhead associated with on-board DRAM memory and garbage collection. However, while background read refreshing serves as a data protection mechanism in conventional block-interface SSDs, state-of-the-art ZNS SSDs lack read refreshing functionality to guarantee data reliability. Moreover, implementing zone-level read refreshing in ZNS SSDs incurs significant overhead due to the large volume of valid data movements in a zone, leading to degraded I/O performance.**

To efficiently enable read refreshing for ZNS SSDs, this paper proposes FineRR-ZNS, a fine-granularity read refreshing mechanism for ZNS SSDs. FineRR-ZNS employs a host-controlled fine-granularity read refreshing scheme that selectively determines block-level read refreshing via metadata remapping. A zone reconstruction method is also designed to retrieve remapped data forming complete data during zone-level RR. Specifically, the remapped data after zone reconstruction are still available and prioritized for read access until their respective blocks need the next RR. Evaluation results show that FineRR-ZNS significantly enhances read refreshing efficiency and I/O throughput compared to zone-level read refreshing implemented in the state-of-the-art ZenFS file system.

***Index Terms*—ZNS SSDs, Fine-granularity read refreshing, Remap, Reconstruct, Replica.**

I. Introduction

NAND flash-based solid-state drives (SSDs) have become the mainstream storage media thanks to their advantages of high access throughput [1]. Due to the nature of out-of-place updates, SSDs require a DRAM cache to expedite translations between logical and physical addresses [2]. Then, garbage collection (GC) is employed to reclaim the available space through valid data movements and block erasures, heavily postponing I/O processing [3]. Additionally, the user-invisible flash capacity, known as over-provisioning (OP) space is reserved, to enhance GC efficiency and availability [4].

To eliminate the DRAM and flash costs and mitigate the GC impacts on I/O processing, a new storage interface, zoned namespace (ZNS) is introduced for flash-based SSDs [5]. Since ZNS interface only supports sequential writes, coarse metadata are maintained, with a zone-level mapping table in file system and a zone-to-block one in ZNS SSDs. This indicates the requirement of DRAM cache inside SSDs can significantly be reduced. Since the host takes charge of the basic operations of read, write and GC, also called zone reset in ZNS SSDs, OP spaces can be removed that are needed by the in-device garbage collection of conventional SSDs [6]. Besides, as application data with similar lifetime are gathered together in the same zone by the host, some fully-invalid zones can be reclaimed directly without heavy data movements [7].

*Corresponding authors: Jieming Yin and Jianwei Liao.

Different from garbage collection, read refreshing, also called read reclaim (RR) is a data rewriting method fighting against read disturb that is a physical circuit noise phenomenon [8]. In other words, while the primitive purpose of garbage collection is to reclaim the space, read refreshing is employed to ensure the data reliability. Therefore, the target of read refreshing is generally full of valid data [9]. To the best of our knowledge, state-of-the-art ZNS-compatible file systems, such as ZenFS [10] and F2FS [11], lack support for host-controlled background data refreshing mechanisms. However, several inherent circuit-level noises, such as retention disturb and read disturb still exist [12].

To ensure the data reliability, the background read refreshing method remains necessary in the context of ZNS SSDs. This paper first explores introducing read refreshing functionality into ZNS SSDs to ensure data reliability. Since zone-granularity management in ZNS SSDs induces a large number of valid data movements during RR operations and thus aggravates I/O delays, FineRR-ZNS is proposed to enable a fine-granularity read refreshing method for ZNS SSDs. It can selectively adopt block-level RR by *Zone Remapping* module, and later combine remapped data and original data by *Zone Reconstruction* module. Furthermore, frequently-read remapped data can be kept as replicated data until its block reaches the next RR threshold. Consequently, FineRR-ZNS can eliminate data movements during RR and result in better I/O performance. This paper offers the following contributions:

- We introduce read refreshing method for ZNS SSDs, to ensure data reliability induced by read disturb.
- We develop FineRR-ZNS, a fine-granularity read refreshing management at the host side for ZNS SSDs, which can eliminate the total number of valid data migrations during read refreshing, thus improving I/O performance.
- We design two functionalities of zone remapping and zone reconstruction to implement FineRR-ZNS without noticeable overhead.
- We conduct experiments on FEMU SSD emulator using RocksDB applications. Compared to zone-management read refreshing method implemented in state-of-the-art Zenfs, FineRR-ZNS can improve RR efficiency and I/O throughput by 36.4% and 28.2% on average, respectively.

II. Background Knowledge

Due to the nature of out-of-place updates, SSDs utilize flash translation layer (FTL) within their internal controller to allocate data to expected location in flash arrays, and maintain a mapping table for the relationship between physical and logical addresses [13]. To reclaim the space occupied by outdated data marked as invalid pages, garbage collections (GCs) are triggered when available space becomes insufficient [14]. Specifically, all the valid data in the

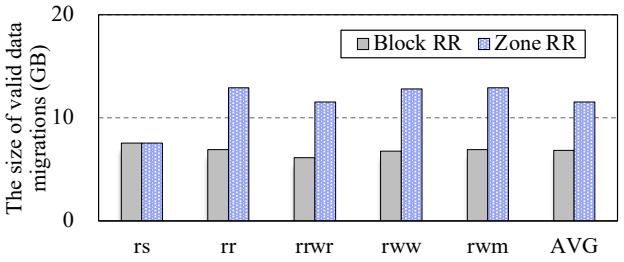

Fig. 1. Comparison on valid data migrations between zone-level and block-level read refreshing.

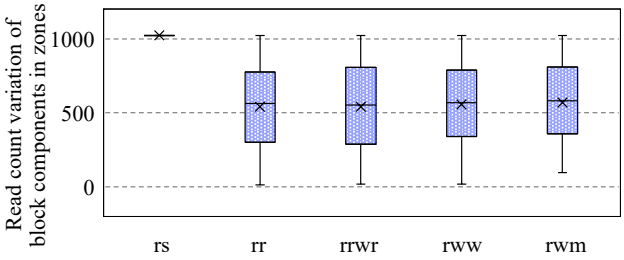

Fig. 2. Box plots of block read count variations during zone-level read refreshing in ZoneRR-ZNS.

selected block are copied to new locations, and then erases and returns the block to an available free state [15]. However, garbage collection remains a persistent challenge in block-interface SSDs, as it is handled by SSDs and transparent to the host [16]. This makes it difficult for the host to predict SSD performance. To mitigate the negative impact of garbage collection, ZNS has been introduced into SSDs, shifting the main software components to the host [17]. Consequently, the host provides software-defined capabilities that handles data placement and I/O scheduling, while SSDs are only responsible for basic read, write, and erase operations [18].

Since ZNS SSDs only support strict sequential writes, a coarse-grained zone-to-block mapping table is maintained inside ZNS SSD device that eliminates DRAM space over than 10X [19], rather than a page-level mapping table. Additionally, valid data migrations with multiple reads and writes during garbage collections are controlled by host, ensuring performance predictability of SSDs. Consequently, the reserved over-provisioning space can also be removed, which comprising 7% to 28% of the advertised storage capacity, further decreasing the SSD cost [5].

In ZNS SSDs, the host manages the data placement at the granularity of zone, comprising a large number of parallel unit with flash blocks. Considering the files with the same lifetime are preferred to be placed in the same zone, the valid data movements can be minimized in ZNS SSDs during garbage collections [20]. However, hot read data in the specific zone accumulate over time, leading to read disturbance issues. Additionally, a high concentration of valid data in a zone significantly impacts I/O efficiency during data refreshing. To address these challenges, we then conduct a preliminary study on read refreshing in ZNS SSDs and analyze the observations.

III. Preliminary Study and Motivation

The background read refreshing method is commonly employed in conventional SSDs, with its threshold typically set based on the worst-case error scenario [21], [22], [23]. For example, the SLC cell has strong reliability, while QLC cell is prone to several disturb-induced errors. Consequently, the block read threshold of read refreshing is correspondingly reduced from 100K to 1K [9], [24]. Since the QLC is widely used in commercial products [25], this paper considers QLC as the default flash media. Considering that the current framework of ZNS SSD system does not support read refreshing module in both file system and SSD controller, we first do a preliminary study on how to implement the read refreshing method for ZNS SSDs.

A. Preliminary Study on Read Refreshing in ZNS SSDs

SSD-side block-level refreshing. An intuitive approach is to implement a read refreshing module directly within SSDs, similar to traditional block-based SSDs. Specifically, it requires both the over-provisioning and DRAM cache spaces for managing the data mapping inside SSDs. However, this approach goes against the foundational intent underlying ZNS design. This indicates that additional over-provisioning and metadata spaces and non-transparent internal operations of SSDs are employed in this design. Considering this approach in turn increases both DRAM and flash cost of ZNS SSDs and degrades performance predictability, it is excluded from consideration in this paper.

Host-side zone-level refreshing. Following the basic principle of ZNS SSDs, the host takes charge of most of software modules. Consequently, the baseline refreshing method is employed as a part of ZNS-aware file system at zone granularity, called *ZoneRR-ZNS*, and we use this method as baseline read refreshing. Specifically, it recalls the functionality of garbage collection, awaking a separated thread termed as *RRWorker*. Note that, since the host does not know the information below zone granularity, a block-level read counter for triggering read refreshing must be kept within SSD controller, requiring around tens of kilobytes that will be described in Section V-C. When the SSD responds with data to the host, it also includes a refreshing flag. If the flag is on, RRWorker is activated to perform valid data migrations and zone resets.

B. Motivational Experiments

We implemented ZoneRR-ZNS in FEMU [26] by running applications of RocksDB [27] with the ZNS-compatible file system plugin of ZenFS [10]. The read refreshing threshold of block read count is set as 1,024, 40 times of the number of pages per block, referred to [24]. The detailed specifications will be described in Section V-A. Once the read refreshing is triggered, we collected the statistic for all blocks within the zone (labeled as Zone RR), and tracked the block-level read refreshing regardless of ZNS write constraints (Block RR). As presented in Figure 1, the benchmark of *readseq (rs)* has entirely sequential reads, so that the zone-level RR does not amplify valid data movements in this scenario. More importantly, we can observe that zone-level read refreshing results in an average of 70% more valid data migrations compared to block-level read refreshing. To clearly show the root cause of increased data movements, we further collect block read count variations across zones in ZoneRR-ZNS.

We utilize box plots to illustrate the variations in block read counts across zones. As shown in Figure 2, the large range of block read counts can indicate that while the current RR block has reached the read count threshold, most blocks are still far from it. In other words, a major part of blocks within the zone do not require immediate data refreshing at that time.

Motivated by aforementioned observations from preliminary studies on ZoneRR-ZNS, this paper proposes a novel read refreshing method for ZNS SSDs, aiming at eliminating the number of concen-

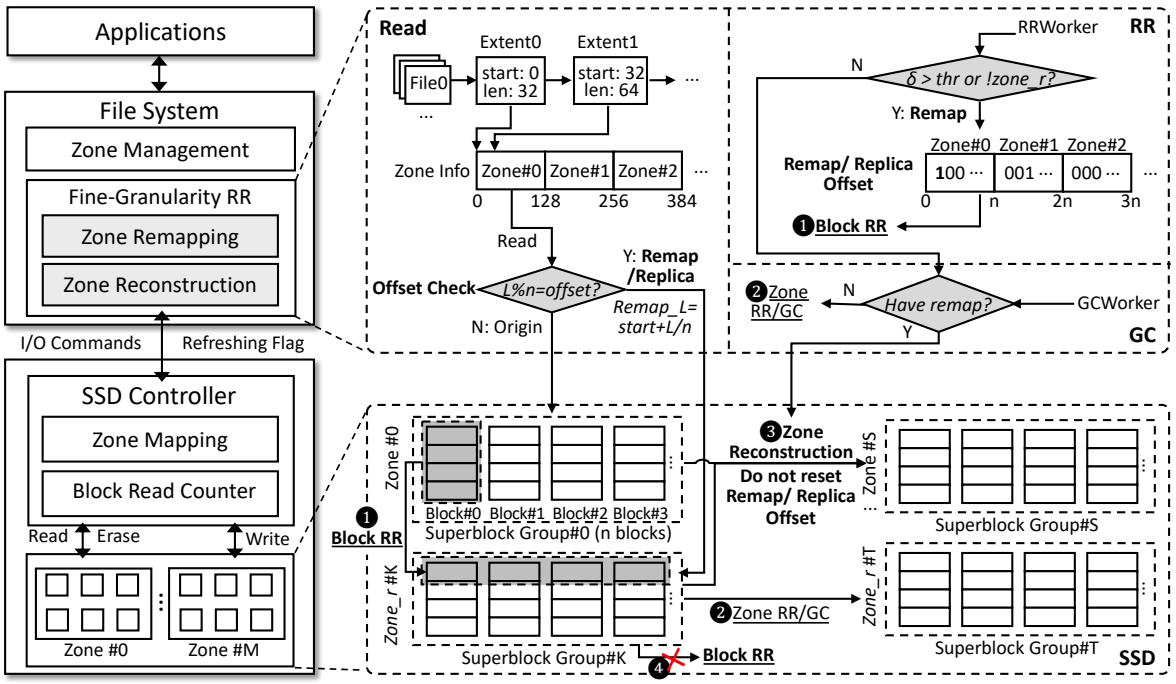

Fig. 3. The overview of FineRR-ZNS, designed for supporting a fine-granularity read refreshing (RR). During read refreshing is required, it is checked by *Zone Remapping* module for employing ❶*Block RR*. When data are required for read requests, they are also checked for fetching data from original zone or remapped zone. Once RR or garbage collection (GC) targets at original zone whose data were previously remapped, ❸*Zone Reconstruction* module is triggered. Note that default ❷zone-level RR and GC is employed if remapping is not enabled, and the second remapping on zone_r is not permitted (❹).

trated data movements during RR operations, eventually enhancing overall I/O performance.

IV. FineRR-ZNS Design

A. Overview

We propose FineRR-ZNS, a fine-granularity read refreshing management for ZNS SSDs. Figure 3 presents the overview of FineRR-ZNS, highlighting its design architecture and illustrating its functionalities. Since blocks within a zone do not necessarily reach the read refreshing threshold simultaneously, FineRR-ZNS selectively refreshes only the required blocks by *zone remapping* module, thereby avoiding unnecessary data movements for other blocks within the zone. It can minimize data movements during RR, thus mitigating I/O congestion and improving I/O throughput. In order to maintain the metadata of remapped data without complex data structures, *zone reconstruction* module is introduced to combine remapped data with original data into the normal zone when the original zone needs refreshing. Additionally, the remapped data are retained as *replicated data*, that are prioritized for read accesses. These data will eventually be directly invalidated without migrations until their block reaches the RR threshold. Consequently, both data movements during RR and I/O congestion caused by RR can be minimized.

B. Benefit Analysis of FineRR-ZNS

The basic intent of FineRR-ZNS is to minimize unnecessary read refreshing operations and mitigate the postpone in I/O processing. Consequently, a block-level read refreshing is supported by zone remapping module that will be detailedly described in Section IV-C. However, when the original zone needs doing zone-level read refreshing, the remapped data by previous block-level RR should also be recovered into sequential data by zone reconstruction module. That is to say, the **cost** of FineRR-ZNS is the data movement number of remapped data in RR block. Fortunately, many unnecessary data movements in zone-level read refreshing that were shown in Figure 1 can be eliminated. Furthermore, since the flash blocks with remapped data can endure additional read accesses without requiring immediate refreshing until reaching their RR threshold, the **benefits** include that RR of other blocks with low read counts can be delayed, and the remapped data can endure further read accesses. Therefore, we can define the expectation of cost-benefit model for data movements by RR operations as follows.

$$\begin{aligned} E &= (\sum_{i=0}^{n-1} \frac{thr - RC_i}{thr} \times N_{page} + \frac{thr}{thr} \times N_{page}) - N_{page} \\ &= \sum_{i=0}^{n-1} \frac{thr - RC_i}{thr} \times N_{page} \end{aligned} \tag{1}$$

where thr is the RR threshold, N_{page} is the page number per block, and RC_i is the read count of ith block in a zone. Note that, $\frac{thr}{thr} \times N_{page}$ is one of the benefit by the remapped data, since it can endure the read count of thr as replica data, though the zone reconstruction is performed. Thus, E is always a non-negative value, and can be considered as a decreasing function of $\sum_{i=0}^{n-1} RC_i$. In other words, FineRR-ZNS enables block-granularity RR when a certain number of blocks in a zone do not reach the refreshing threshold. To support FineRR-ZNS, zone remapping management and zone reconstruction management are designed in §IV-C and §IV-D, respectively.

C. Zone Remapping

Similar to the baseline method of ZoneRR-ZNS described in Section III-A, a separated thread RRWorker in file system and a block read counter in SSD controller are utilized for implementing

RR. Apart from a zone-level read refreshing (❷zone RR) flag in ZoneRR-ZNS, FineRR-ZNS introduces individual block flags within a zone for block-level read refreshing (❶block RR). This section will describe **when** to determine block RR and **how** to conduct it in ZNS SSDs.

Determining block RR. Once read refreshing is required, FineRR-ZNS determines whether to perform zone-level or block-level read refreshing. Motivated by the observation in Figure 2 and the benefit analysis in Equation 1, FineRR-ZNS can improve the space wastage induced by zone RR when the other blocks are far from the RR threshold. Thus, we use $\sum_{i=0}^{n-1}(thr - RC_i)$ as an indicator δ to enable the block RR. Considering that a small value of δ may not notably benefit RR operations, the threshold for δ defined as equal to the RR threshold, set to 1,024 in the experimental setting.

Handling block RR. Once block RR is required, zone remapping module in FineRR-ZNS migrates the data to the specific zone, such as *Zone #K* as shown in Figure 3, and modified the flag of refreshed flash block in its zone in *remap offset*. Importantly, zone remapping module maintains the metadata of remapped data. It holds offset of remapped block as a bit map for checking $L\%n = offset$, as well as the start address of remapped zone, denoted as $start$. Consequently, the remapping address can be calculated by $Remap\_L = start + L/n$, where L is the original logical address of the request, and $start$ is the starting address of the remapped data. The unrefreshed data can be accessed in the original zone as default. That is to say, a read request requires checking the remapping information. If the remapping offset is matched, the data should be fetched from the remapped zone, termed as *zone_r*, based on their addresses and offsets. Otherwise, the default read mode is employed. Note that the frequently read data with block granularity have been distributed in different flash blocks in zone_r, so that we argue that the read accesses on zone_r are uniformly distributed across blocks.

To facilitate the management of remapped data, the data should be relocated to the dedicated zone. When the zone remapping is enabled, it first checks whether the left space in the current open zone_r is enough. If not, it applies for a new zone from free zone list and assigns it as zone_r.

D. Zone Reconstruction

When the original zones need to carry out zone-level RR or GC, the complete data, including the remapped data must be integrated. Otherwise, several discontinuous data left in the original zone cannot be migrated to another zone due to inconsistency with ZNS writes. In our design of FineRR-ZNS, ❸zone reconstruction is enabled when the original zones having remapped data are selected as targets of zone-level RR or GC. If activated, zone reconstruction is employed by retrieve all the associated remapped data and then merge them with the original data.

Although zone remapping module can reduce data movements during a given RR operation compared to zone-level RR, FineRR-ZNS utilizes the more space to additionally occupy the zone_r regions, as analyzed in Equation 1. To eliminate the space occupation by remapped data, zone reconstruction module not only reorganize the remapped and original data, but also keeps the remapped data in a valid state for further accesses. Thus, the remap offset also serves as the *replica offset* without resetting to 0 until the replicated data blocks reach the RR threshold.

Additionally, in order to simplify the remapping metadata management, the second remapping on the same data is not permitted, as shown in ❹ in Figure 3. We argue that the frequently read data is relatively uniform within zone_r, as the remapped data have been distributed from individual blocks to multiple blocks, eliminating the need for another block RR in remapped data zones. Consequently, only the data in normal zone can be remapped by FineRR-ZNS, while those in zone_r can only be reconstructed with original data (❸) or be employed in zone-level RR and GC (❷). Note that the zone-level RR or GC on zone_r migrates data to another zone_r, and the start address *start* should be updated correspondingly.

TABLE I
EXPERIMENTAL SETUP

Hardware Platform	
(FEMU, Kernel Version)	*(8.0, 5.10.0)*
(Channel, Chip, Die Plane, Block, Page)	*(8, 2, 2, 4, 256, 256)*
(Page Size, Cell Density)	*(4KB, QLC)*
(Read, Program, Erase Latency)	*(40μs, 200μs, 2ms)*
Software Platform	
(RocksDB, ZenFS Version)	*(7.7.3, 2.0.1)*
(Zone size, Number of Zones)	*(128MB, 256)*
(GC, RR Threshold)	*(20%, 1,024)*

V. EXPERIMENT AND EVALUATION

A. Experiment Setup

We evaluate our proposed method by using FEMU [26], a QEMU-based SSD emulator that supports ZNS SSD functionality and full system stack, which has been widely utilized for recent ZNS SSD researches [28], [7], [29], [30]. It runs Rocksdb applications [31] with Zenfs in-application file system [10]. The detailed specifications of hardware and software platforms are shown in Table I. Specifically, the flash parameters are based on [30] and zone settings are followed by [28]. Each superblock comprises 128 blocks having same offset across all flash planes, with both the zone size and superblock size configured to 128MB. The thresholds of garbage collection and read refreshing are referred to [10] and [24], respectively.

Additionally, a built-in benchmark tool *db_bench* of RocksDB is used to generate different workloads [27]. We evaluate 5 kinds of workloads, each with 60 million key-value pairs of 1KB size, referred to [19]. Specifically, *readseq (rs)*, *readrandom (rr)*, *readrandomwriterandom (rrwr)*, *readwhilewriting (rww)*, *readwhilemerging (rwm)* are used for evaluating effectiveness of the proposed method.

We employ the following comparison counterparts to validate the effectiveness of our proposal.

- *NoRR-ZNS*, which is a pure ZNS SSD with Zenfs, without enabling read refreshing method. We set this method as a comparison to show the upper limit of I/O performance, though it cannot ensure the data reliability.
- *ZoneRR-ZNS*, which has been described in Section III. A host-controlled zone-level read refreshing method is implemented via calling the functionality of garbage collection.
- *FineRR-ZNS*, which is the new proposed method. It supports fine-granularity read refreshing for ZNS SSDs, implemented by two modules of remapping and reconstruction.

B. Evaluation and Result

To measure the validity of the proposed mechanism that aims to improve RR efficiency and boost I/O performance for ZNS SSDs, we use the following metrics in our tests: (a) *RR efficiency* including valid data movements and erase counts, and (b) *I/O performance* to measure the quality of service.

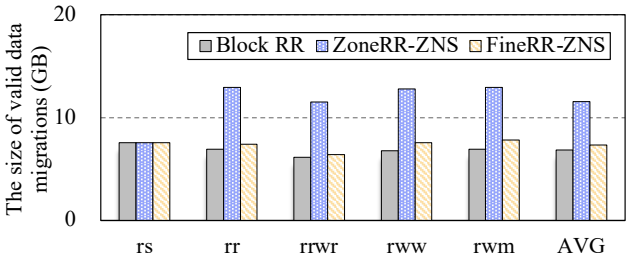

Fig. 4. The comparison of valid data movement during read refreshing after running selected RocksDB workloads.

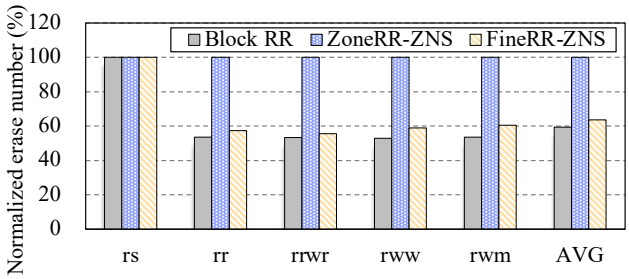

Fig. 5. The comparison of block erasure number after running selected RocksDB workloads.

1) RR efficiency: The basic idea of FineRR-ZNS is to avoid unnecessary valid data movement in RRs, and the statistic on it is shown in Figure 4. Except for similar performance in benchmark of *rs* with full sequential reads, FineRR-ZNS significantly reduce the amount of valid pages by 41.8% on average, compared with ZoneRR-ZNS. This is because FineRR-ZNS offers a fine-granularity read refreshing, eliminating the most of data migrations induced by default zone-level read refreshing, as analyzed in Equation 1.

Additionally, we also include *Block RR* that was presented in Section III-B, to show the lower limit of RR statistics, though it does not obey the sequential write constraint of ZNS interface. As seen, the performance of FineRR-ZNS in valid data movements is approaching the lower limit presented in Block RR. The remaining 7.1% of data movements result from two factors: (a) the threshold set for enabling block-level RRs, which avoids excessive and performance-inefficient block-level operations but inevitably induces a few more data movements during zone-level RR, and (b) the replicated data retained in the remapping zone, which not yet endure enough read accesses at the end of evaluated benchmarks, leading to additional data migrations.

The erasure count is a key metric reflecting SSD lifetime, due to the erase limit of flash memory. Figure 5 presents the results of erasure count induced by read refreshing operations. As unveiled, FineRR-ZNS can notably decrease the number of block erasure by an average of 36.4% compared with ZoneRR-ZNS. This is because FineRR-ZNS can effectively eliminate the valid data movements, resulting efficient block erasure.

2) I/O Performance: FineRR-ZNS can improve RR efficiency and thus mitigate delays in I/O processing caused by excessive RRs. I/O throughput is an indicator to reflect the efficiency of FineRR-ZNS, as seen in Figure 6. As shown, FineRR-ZNS can improve data access throughput by 28.2% on average, compared with ZoneRR-ZNS. We also include the NoRR-ZNS method that is not enabled read refreshing method, to show the upper limit of the scenarios without

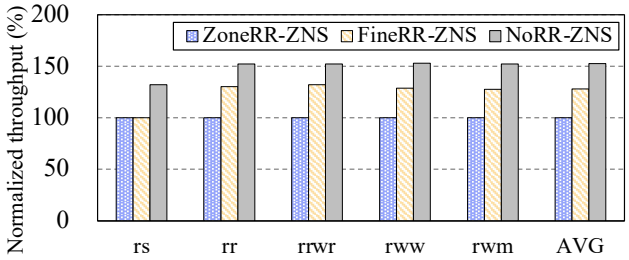

Fig. 6. The comparison of I/O throughput after running selected RocksDB workloads.

TABLE II
ADDITIONAL SPACE OVERHEAD FOR 32GB ZNS SSD REQUIRED IN ZONERR-ZNS AND FINERR-ZNS.

Methods	*ZoneRR-ZNS*	*FineRR-ZNS*
SSD overhead	40KB	40KB
Host overhead	0.032KB	68.07KB

delays by RR operations. But it is important to note that NoRR-ZNS cannot guarantee data reliability, since read disturb may lead to data loss. Even considering the NoRR-ZNS method, FineRR-ZNS unveils attractive I/O performance with the data protection mechanism of RR enabled.

C. Overhead Analysis

Metadata overhead. The space overhead mainly consists of remapping metadata to enable fine-granularity read refreshing in ZNS SSDs. The detailed space overhead is presented in Table II. To deploy block-level counters in SSD, both ZoneRR-ZNS and FineRR-ZNS need 40 kilobytes. Additionally, they requires a refreshing flag to trigger RR operation in host, and FineRR-ZNS needs an additional flag to direct whether block- or zone-level refreshing based on δ. As for remapping metadata, remap offset needs flags of each block, and the start address of remapped zone *start* is recorded by zone ID and zone offset, involving 4+4 bits for respectively representing 256 zones and 256 pages. In total, ZoneRR-ZNS needs 1-bit flag for each zone, while FineRR-ZNS requires 2 bits for determining RR, 128 bits for remap/replica offset table, and additional 128×16 bits for remapped start address. Fortunately, these overheads are negligible when managed by the host.

Time overhead. FineRR-ZNS only adds a few judgments to determine the original mapping or zone remapping. Additionally, FineRR-ZNS needs computing δ. Because δ is calculated through several addition and subtraction operations, its computing overhead is minimal. In summary, FineRR-ZNS has negligible additional time overhead to enable fine-granularity read refreshing for ZNS SSDs.

VI. RELATED WORK

Garbage collection optimizations in ZNS SSDs. Since the space reclaim granularity of ZNS SSDs is zone, several researches focus on garbage collection optimizations. Choi et al. [32] proposed to separate hot and cold data during garbage collections, and thus the later GC efficiency can be enhanced. Huang et al. [29] introduced smaller zone size to reduce data migration in garbage collections, and proposed several optimization methods for alleviating the degraded performance due to underutilized parallelism. Long et al. [28] proposed WA-Zone to avoid unnecessary erases upon unused blocks and balance inter- and intra-zone wear in ZNS SSDs. However, the

background read refreshing method is not employed for ZNS SSDs for ensuring the reliability of ZNS SSDs. This paper proposes to implement the read refreshing method for ZNS SSDs and optimize it via fine granularity.

Read refreshing optimizations in conventional SSDs. The read refreshing method is a background method in conventional SSDs. Since a large number of data movements block the front-end I/O responses in SSDs, several methods proposed to manage the frequently read data in dedicated blocks. Liu et al. [33] proposed *Read Leveling*, a strategy that allocates specific blocks, known as shadow blocks, to mitigate the effects of read disturb. Since the blocks contain several invalid data pages or free pages, both of which are unaffected by read disturb, Read Leveling only refreshes the frequently read accessed data in shadow blocks. Additionally, Wu et al. [34] proposed to reprogram TLC blocks to MLC blocks, since MLC blocks can enable a larger margin between different states to tolerate read disturb. However, these methods decrease the space utilization, caused by sacrificing available dedicated blocks to fight against read disturb.

For further eliminating the negative impacts of read refreshing towards I/O response and optimizing the read data allocation management, Li et al. [35] introduced reinforcement learning to utilize idle intervals for partial read refreshing, mitigating the postpones on I/O responses. Zhao et al. [36] optimized data allocation by estimating optimal read limits based on initial read counts and P/E cycles of flash blocks, that can eliminate the frequency of read refreshing operations and ECC time overhead. Zhang et al. [37] proposed redistributing hot read data across multiple blocks with cold data to limit read concentration, thereby decreasing RR frequency.

These optimization methods target at the conventional block-interface SSDs. In ZNS SSDs, enlarged zone-level management and sequential programming management prevent enabling efficient read refreshing, and our proposal of fine-granularity read refreshing can significantly improve RR efficiency and I/O processing.

VII. Conclusion

This paper proposes an efficient read refreshing for ZNS SSDs to fight against read disturbs. The primary objective is to eliminate unnecessary valid data movements under zone-level management. To this end, a fine-granularity read refreshing method is designed with two key components of zone remapping and zone reconstruction. Consequently, unnecessary data migrations by zone-level read refreshing can be significantly reduced, thereby mitigating the I/O postpones. Evaluation results measure that the proposed method can yield attractive improvement on I/O throughput by an average of 28.2% and space utilization by up to 44.4% compared to zone-level read refreshing scheme.

Acknowledgment

We sincerely thank the reviewers for their valuable suggestions. This work was partially supported by National Key R&D Program of China Grant No. 2023YFB4404400, National Natural Science Foundation of China Grants No. 62102193, No. 92473205, No. 61872299, State Key Laboratory of Computer Architecture (ICT CAS) Grant No. CARCH202106, and Natural Science Research Start-up Foundation of Recruiting Talents of Nanjing University of Posts and Telecommunications Grant No. NY224062.

References

[1] L. Zuolo, C. Zambelli, R. Micheloni *et al.*, "Solid-state drives: Memory driven design methodologies for optimal performance," *Proceedings of the IEEE*, vol. 105, no. 9, pp. 1589–1608, 2017.

[2] R. Micheloni, "Solid-state drive (ssd): A nonvolatile storage system," *Proceedings of the IEEE*, vol. 105, no. 4, pp. 583–588, 2017.

[3] T. Lange, J. Naor, and G. Yadgar, "Offline and online algorithms for ssd management," *Communications of the ACM*, vol. 66, no. 7, pp. 129–137, 2023.

[4] F. Wu, J. Zhou, S. Wang *et al.*, "Fastgc: Accelerate garbage collection via an efficient copyback-based data migration in ssds," in *Proceedings of the 55th Annual Design Automation Conference (DAC)*, 2018, pp. 1–6.

[5] M. Bjørling, A. Aghayev, H. Holmberg *et al.*, "ZNS: avoiding the block interface tax for flash-based ssds," in *USENIX Annual Technical Conference (ATC)*, 2021.

[6] K. Han, H. Gwak, D. Shin *et al.*, "ZNS+: advanced zoned namespace interface for supporting in-storage zone compaction," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021.

[7] Y. Wang, Y. Zhou, Z. Lu *et al.*, "Flexzns: Building high-performance zns ssds with size-flexible and parity-protected zones," in *2023 IEEE 41st International Conference on Computer Design (ICCD)*. IEEE, 2023, pp. 291–299.

[8] K. Ha, J. Jeong, and J. Kim, "An integrated approach for managing read disturbs in high-density nand flash memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 7, pp. 1079–1091, 2015.

[9] J. Liao, J. Li, M. Zhao *et al.*, "Read refresh scheduling and data reallocation against read disturb in ssds," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 21, no. 2, pp. 1–27, 2022.

[10] Western Digital Corporation, "Zenfs github repository," https://github.com/westerndigitalcorporation/zenfs, 2024.

[11] C. Lee, D. Sim, J. Hwang *et al.*, "{F2FS}: A new file system for flash storage," in *13th USENIX Conference on File and Storage Technologies (FAST 15)*, 2015, pp. 273–286.

[12] Y. Luo, S. Ghose, Y. Cai *et al.*, "Improving 3d nand flash memory lifetime by tolerating early retention loss and process variation," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, pp. 1–48, 2018.

[13] W. Zhang, Q. Cao, H. Jiang *et al.*, "Pa-ssd: A page-type aware tlc ssd for improved write/read performance and storage efficiency," in *Proceedings of the 2018 International Conference on Supercomputing*, 2018, pp. 22–32.

[14] C. Gao, L. Shi, Y. Di *et al.*, "Exploiting chip idleness for minimizing garbage collection—induced chip access conflict on ssds," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 23, no. 2, pp. 1–29, 2017.

[15] J. Cui, Y. Zhang, J. Huang *et al.*, "Shadowgc: Cooperative garbage collection with multi-level buffer for performance improvement in nand flash-based ssds," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1247–1252.

[16] J. Min, C. Zhao, M. Liu *et al.*, "{eZNS}: An elastic zoned namespace for commodity {ZNS}{SSDs}," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 461–477.

[17] D. Seo, P.-X. Chen, H. Li *et al.*, "Is garbage collection overhead gone? case study of f2fs on zns ssds," in *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*, 2023, pp. 102–108.

[18] T. Kim, J. Jeon, N. Arora *et al.*, "Raizn: Redundant array of independent zoned namespaces," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 660–673.

[19] Y. Wang, Z. Sun, Y. Zhou *et al.*, "Balloon-zns: Constructing high-capacity and low-cost zns ssds with built-in compression," in *Design Automation Conference (DAC)*, 2024.

[20] Q. Wang, J. Li, P. P. Lee *et al.*, "Separating data via block invalidation time inference for write amplification reduction in {Log-Structured} storage," in *20th USENIX Conference on File and Storage Technologies (FAST 22)*, 2022, pp. 429–444.

[21] Micron Technical Note (No. TN-29-27): Design and Use Considerations for NAND Flash Memory.

[22] J. Werner, E. T. Cohen, and T. L. Canepa, "Read disturb handling for non-volatile solid state media," May 15 2014, uS Patent App. 13/729,966.

[23] Y. Seo, J. Yun, W. Lee *et al.*, "Memory controller, method of operating the same and memory system including the same," Dec. 13 2016, uS Patent 9,519,576.

[24] C. Liu, Y. Lee, M. Jung *et al.*, "Prolonging 3d NAND SSD lifetime via read latency relaxation," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
[25] Q. Li, H. Dang, Z. Wan *et al.*, "Midas touch: Invalid-data assisted reliability and performance boost for 3d high-density flash," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 657–670.
[26] Huaicheng Li, "Femu github repository," https://github.com/MoatSysLab/FEMU, 2024.
[27] Z. Cao, S. Dong, S. Vemuri *et al.*, "Characterizing, modeling, and benchmarking {RocksDB}{Key-Value} workloads at facebook," in *18th USENIX Conference on File and Storage Technologies (FAST 20)*, 2020, pp. 209–223.
[28] L. Long, S. He, J. Shen *et al.*, "Wa-zone: Wear-aware zone management optimization for lsm-tree on ZNS ssds," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 21, no. 1, pp. 16:1–16:23, 2024.
[29] D. Huang, D. Feng, Q. Liu *et al.*, "Splitzns: Towards an efficient lsm-tree on zoned namespace ssds," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 3, pp. 1–26, 2023.
[30] Z. Tan, L. Long, J. Shen *et al.*, "Optimizing garbage collection for zns ssds via in-storage data migration and address remapping," *ACM Transactions on Architecture and Code Optimization*, 2024.
[31] Facebook, "Rocksdb github repository," https://github.com/facebook/rocksdb, 2024.
[32] G. Choi, K. Lee, M. Oh *et al.*, "A new {LSM-style} garbage collection scheme for {ZNS}{SSDs}," in *USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 20)*, 2020.
[33] C.-Y. Liu, Y.-M. Chang, and Y.-H. Chang, "Read leveling for flash storage systems," in *Proceedings of the 8th ACM International Systems and Storage Conference*, 2015, pp. 1–10.
[34] T.-C. Wu, Y.-P. Ma, and L.-P. Chang, "Flash read disturb management using adaptive cell bit-density with in-place reprogramming," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 325–330.
[35] J. Li, B. Huang, Z. Sha *et al.*, "Mitigating negative impacts of read disturb in ssds," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 26, no. 1, pp. 1–24, 2020.
[36] M. Zhao, J. Li, Z. Cai *et al.*, "Block attribute-aware data reallocation to alleviate read disturb in ssds," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1096–1099.
[37] G. Zhang, Y. Deng, Y. Zhou *et al.*, "Cocktail: Mixing data with different characteristics to reduce read reclaims for nand flash memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 7, pp. 2336–2349, 2022.